

Chapter 3 - Part II: Constraint Satisfaction Problems (CSPs)

Zehara Eshetu

Department of Computer Science

2026

Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

What are CSPs?

Definition

- CSPs are mathematical problems defined as a set of objects whose state must satisfy a number of constraints
 - A constraint is a limitation on what type of solution we will allow
- **Standard search problem:**
 - So far we have formulated problems by stating: **initial state**, **actions(successor function)**, **goal test**, **path cost** and **heuristic function**
 - We have not so far made any restrictions on allowable solutions, apart from an explicit goal test
- In CSP a problem is formulated by stating:
 - variables X_i with values from domain D_i
 - goal test is a set of constraints specifying allowable combinations of values for subsets of variables, X_i

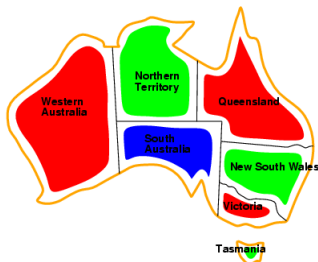
Map-Coloring

- This is a famous example in AI and is often used to illustrate CSPs.
- Suppose we have a map of a country, like the map of Australia as shown below.
 - The country is divided into a number of regions.
- The problem is how we can colour each region of the country using a limited number of colours (in this case 3, RGB) so that no two adjacent regions have the same colour.



Example: Map-Coloring

- **Variables:** WA, NT, Q, NSW, V, SA, T
- **Domains D_i** = red, green, blue
- **Constraints:** adjacent regions must have different colors, e.g.
 WA NT, or (WA,NT) in
 $\{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\}$

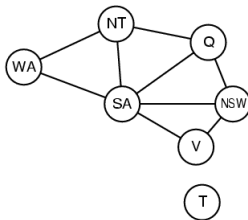


Example: Map-Coloring

- Solutions are **complete** and **consistent** assignments.
 - **Complete:** one in which all variables have been assigned a value
 - **Consistent:** one that does not violate any of the specified constraints
- E.g., WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green

Constraint Graph

- **Constraint graph:** used to visualize CSPs. nodes are variables, edges (arcs) are constraints
- Graph can be used to simplify search.



- **Binary CSP:** each constraint relates two variables

Varieties of CSPs

- It is possible to categorize CSPs based on the type of variable they use and on the size of their domains:
- **Discrete CSPs:** variables take on discrete values
 - finite domains: the red, green, blue domain in the map-colouring problem
 - infinite domains: integers, strings, etc.
- **Continuous CSPs:** variables can take one from a continuous range of values E.g. real numbers are continuous
- Finite domain discrete variables are the easiest type

Varieties of CSPs

- We can also categorize CSPs based on the type of their constraints
 - **Unary** constraints involve a single variable. E.g., $SA \neq \text{green}$
 - **Binary** constraints involve pairs of variables. E.g., $SA \neq WA$
 - **Higher-order** constraints involve 3 or more variables
- The more variables that are involved in a constraint, the harder the problem is to solve.
- Problems with higher-order constraints are the hardest class of problems

Real-world CSPs

- Timetable scheduling
 - E.g., Which class is offered when and where? Who teaches what class?
- Transportation scheduling
 - Assignment of vehicle to drivers/jobs
- Factory scheduling
 - Assignment of staff to jobs/machines, deadlines, etc.

Search Techniques for CSPs

- There are **two** ways of formulating any search problem: **complete state** and **incremental**
- **Incremental formulation** - states are defined by values assigned so far:
 - **Initial state:** the empty assignment $\{\}$
 - **Successor function:** assign a value to an unassigned variable that does not conflict with current assignment
 - fail if no legal assignments
 - **Goal test:** the current assignment is complete
- **Complete-state formulation** (every state is a complete assignment, but may not meet constraints)
 - **Initial state** is a (random) complete assignment
 - **Successor function** modifies 1 variable to try to meet more constraints
 - **Goal test:** the current assignment is consistent
- The search technique that we use for CSPs depends on what formulation we use:
 - Incremental formulation $\rightarrow$ Backtracking search
 - Complete-state formulation $\rightarrow$ Local search

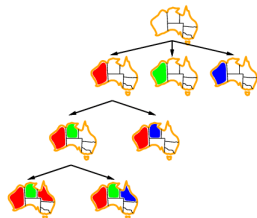
Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Backtracking Search

- The term backtracking search is used for a depth-first search that:
 - chooses values for one variable at a time and
 - backtracks when a variable has no legal values left to assign.
- Variable assignments are **commutative**, i.e.,
[WA = red followed by NT = green] is the same as
[NT = green followed by WA = red]

Backtracking Example



Improving Backtracking Efficiency

- **General-purpose** methods can give huge gains in speed:
 - Which variable should be assigned next?
 - In what order should its values be tried?
 - Can we detect inevitable failure early?
- These factors all have a big influence on the efficiency of backtracking search
 - by making the right choices of variable/value we can make dramatic improvements in efficiency.

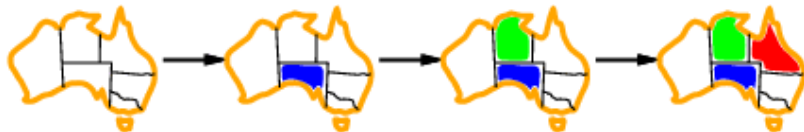
Minimum Remaining Values (MRV)

- Also known as most constrained variable heuristic
- **Rule:** choose the variable with the fewest legal values
- Which variable shall we try first?



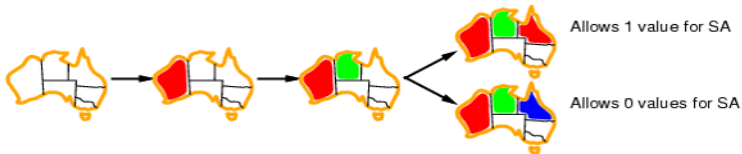
Most Constraining Variable

- Also known as degree heuristic
- **Rule:** select variable that is involved in the largest number of constraints on other unassigned variables.
- Degree heuristic is very useful as a tie-breaker among most constrained variables
 - In what order should its values be tried?



Least Constraining Value

- Given a variable, choose the least constraining value:
 - the one that rules out the fewest values in the remaining variables



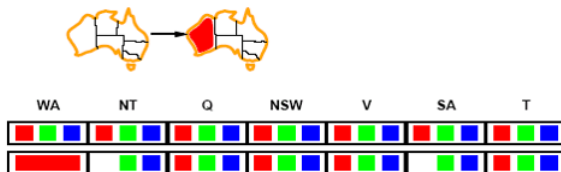
- For example, in the above Figure we have generated the partial assignment with WA=red and NT=green, and that our next choice is for Q. Blue would be a bad choice, because it eliminates the last legal value left for Q's neighbor, SA. The least-constraining-value heuristic therefore prefers red to blue.
- Combining these heuristics makes Backtracking efficient

Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Forward Checking

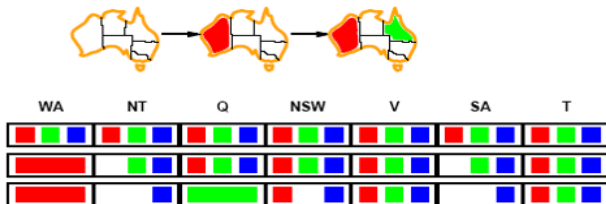
- **Idea:** keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.



- Assign $\{WA = \text{red}\}$
- Effects on other variables connected by constraints with WA
 - NT can no longer be red
 - SA can no longer be red

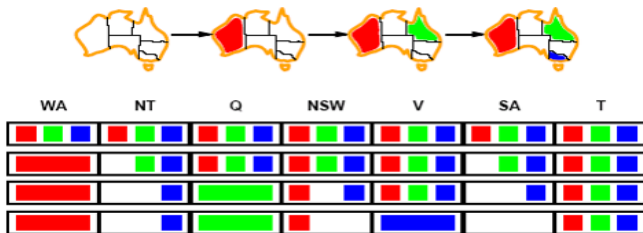
Forward Checking

- Assign $\{Q = \text{green}\}$
- Effects on other variables connected by constraints with WA
 - NT can no longer be green
 - NSW can no longer be green
 - SA can no longer be green



Forward Checking

- If V is assigned blue
- Effects on other variables connected by constraints with WA
 - SA is empty NSW can no longer be blue
- FC has detected that partial assignment is inconsistent with the constraints and backtracking can occur.



Outline

- ① Constraint Satisfaction Problems (CSPs)
- ② Backtracking Search
- ③ Forward Checking
- ④ Local search for CSPs

Local search for CSPs

- For **complete-state** formulation we start with complete states , i.e. all variables assigned
 - Local search techniques typically work with complete states
- To apply to CSPs:
 - allow states with unsatisfied constraints – start with random state
 - operators **reassign** variable values
- Variable selection: randomly select any conflicted variable
- Value selection by **min-conflicts** heuristic:
 - Select new value that results in a minimum number of conflicts with the other variables (choose value that violates the fewest constraints)

Exercises

Consider the map of Ethiopia for the problem of map coloring.

